

Relatório de Melhorias de Segurança

Sistema de Loja de Informática — Python / Tkinter / MariaDB

Data:	30 de April de 2026
Versão:	1.0
Autor:	Equipa de Desenvolvimento
Classificação:	Confidencial — uso interno

0. Introdução

Este documento descreve as duas melhorias de segurança implementadas na aplicação desktop 'Sistema de Loja de Informática'. Ambas as funcionalidades abordam riscos identificados no ciclo de vida da autenticação do utilizador:

- Proteção contra ataques de força bruta no ecrã de login (LoginGuard)
- Expiração automática de sessão por inatividade (SessionManager)

As implementações seguem o princípio do mínimo privilégio, não introduzem dependências externas de segurança desnecessárias e são auditáveis através do sistema de logging já existente no projeto.

1. Proteção Contra Força Bruta — LoginGuard

1.1 Problema identificado

Antes desta melhoria, o ecrã de login não tinha qualquer limite de tentativas. Um atacante podia tentar combinações de palavra-passe indefinidamente (ataque de dicionário / brute-force) sem ser bloqueado.

1.2 Solução implementada

Foi criado o módulo `src/utils/login_guard.py` com a classe `LoginGuard`, que regista as tentativas de login falhadas por endereço de e-mail em memória. Após atingir o limite configurado, a conta fica bloqueada temporariamente.

1.3 Parâmetros de configuração (`src/config.py`)

MAX_LOGIN_ATTEMPTS 5 tentativas
LOGGING_LOCKOUT_SECONDS 300 segundos (5 minutos)

1.4 Fluxo de protecao

- O utilizador submete o formulario de login.
- O LoginGuard verifica se o e-mail esta bloqueado:
 - Se bloqueado => mostra tempo restante e impede a tentativa.
 - Se o bloqueio expirou => desbloqueia automaticamente e permite nova tentativa.
- Se a palavra-passe estiver incorreta => regista a falha e informa as tentativas restantes.
- Se atingir o limite => ativa o bloqueio com timestamp.

5. Em caso de sucesso => reseta o contador de falhas.

1.5 Fragmento de código — verificação no login

```
# src/ui/screens/login.py

if self.guard.esta_bloqueado(email):
    segundos = self.guard.segundos_restantes(email)
    self.notify.error(
        f'Conta bloqueada temporariamente.\n'
        f'Tente novamente em {segundos}s.'
    )
    return

# ... tentativa de autenticação ...

if not verify_password(password, cliente.password):
    self.guard.registar_falha(email)
    tentativas = self.guard.tentativas_restantes(email)
    if tentativas > 0:
        self.notify.error(f'Palavra-passe incorreta! ({tentativas} restante(s))')
    else:
        self.notify.error('Conta bloqueada por excesso de tentativas.')
    return

self.guard.resetar(email) # login bem-sucedido
```

1.6 Persistência do guard entre sessões

O LoginGuard é instanciado UMA ÚNICA VEZ no `__init__` do LojaApp e passado como argumento ao LoginScreen. Desta forma, o contador de falhas persiste mesmo quando o ecrã de login é recriado após logout — impedindo que um atacante contorne o bloqueio fazendo logout e voltando ao ecrã de login.

```
# src/ui/app.py

class LojaApp:
    def __init__(self, master):
        ...
        self.guard = LoginGuard() # criado uma única vez

    def mostrar_login(self):
        login = LoginScreen(..., guard=self.guard) # partilhado
        login.show()
```

1.7 Ficheiros modificados

CRIADO	src/utils/login_guard.py	Classe LoginGuard com toda a lógica
MODIFICADO	src/ui/app.py	Instanciação única + passagem ao LoginScreen
MODIFICADO	src/ui/screens/login.py	Integração nas verificações de login

2. Expiração de Sessão por Inatividade — SessionManager

2.1 Problema identificado

Sem gestão de sessão, um utilizador autenticado que abandonasse o computador deixava a aplicação acessível

indefinidamente. Qualquer pessoa com acesso físico à máquina podia aceder à conta sem qualquer autenticação.

2.2 Solução implementada

Foi criado o módulo `src/utils/session_manager.py` com a classe `SessionManager`. O mecanismo usa o método `after()` do Tkinter — sem threads adicionais — verificando periodicamente (a cada 10 segundos) se o utilizador permanece ativo. Se não houver qualquer interação durante o período configurado, a sessão expira e o logout é executado automaticamente.

2.3 Parâmetro de configuração (`src/config.py`)

SESSION_TIMEOUT_SECONDS 1800 segundos (30 minutos)

2.4 Eventos monitorizados

A aplicação regista os seguintes eventos Tkinter para detetar atividade:

- `<Motion>` — movimento do rato
- `<Key>` — pressão de qualquer tecla
- `<Button>` — clique do rato

2.5 Ciclo de vida da sessão

```
# src/ui/app.py

def on_login_success(self, usuario, db):
    self.session = SessionManager(self.master, on_expire=self._sessao_expirada)
    self.session.iniciar()
    self.master.bind_all('<Motion>', self.session.registar_atividade)
    self.master.bind_all('<Key>', self.session.registar_atividade)
    self.master.bind_all('<Button>', self.session.registar_atividade)

def fazer_logout(self):
    if self.session is not None:
        self.session.parar() # cancela o after() pendente
        self.session = None
```

2.6 Callback de expiração

Quando a sessão expira, o método `_sessao_expirada()` é invocado automaticamente. Este método regista um aviso no log, notifica o utilizador com uma mensagem clara e executa o logout de forma segura.

```
def _sessao_expirada(self):
    logger.warning(
        'Sessao expirada – utilizador: %s',
        self.usuario_atual.email if self.usuario_atual else 'desconhecido',
    )
    self.notify.warning(
        'A sua sessao expirou por inatividade.\n'
        'Por favor, faça login novamente.'
    )
    self.fazer_logout()
```

2.7 Ficheiros modificados

CRIADO	<code>src/utils/session_manager.py</code>	Classe <code>SessionManager</code> completa
MODIFICADO	<code>src/ui/app.py</code>	Integração no ciclo de login/logout

3. Resumo de Impacto de Segurança

Ameaça	Melhoria	Mitigação
Brute-force / dicionário	LoginGuard	Bloqueio após 5 falhas, 5 min de lockout
Sessão abandonada	SessionManager	Logout automático após 30 min de inatividade
Bypass por logout+login	Guard partilhado	Guard persiste no LojaApp, não no ecrã

4. Notas e Limitações

- O LoginGuard mantém os contadores em memória RAM. Se a aplicação for reiniciada, os contadores são resetados. Para persistência entre reinícios, os dados deveriam ser guardados na base de dados.
- O SessionManager usa o scheduler do Tkinter (after). Se a janela principal for destruída abruptamente, o job pode não ser cancelado corretamente. O método parar() deve sempre ser chamado antes de destruir a janela.
- Os valores de timeout e lockout estão centralizados em src/config.py e podem ser configurados via variáveis de ambiente (.env), sem alterar o código fonte.